

```

### NLP NATURAL LANGUAGE PROCESSING
#" I am living in London.
#NLP to make predictions in words by converting them in no.
#bridging gap bw human language and machine understanding

```

```

!pip install nltk
import nltk

# Download essential NLTK datasets (only first time)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')

```

Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.0)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker_tab.zip.
True

```

```

#tokenization(sentence+word)
from nltk.tokenize import sent_tokenize, word_tokenize

text = "Natural Language Processing is amazing. You can analyze text easily!"

# Sentence Tokenization
sentences = sent_tokenize(text)
print("Sentence Tokens:", sentences)

# Word Tokenization
words = word_tokenize(text)
print("Word Tokens:", words)

```

```

Sentence Tokens: ['Natural Language Processing is amazing.', 'You can analyze text easily!']
Word Tokens: ['Natural', 'Language', 'Processing', 'is', 'amazing', '.', 'You', 'can', 'analyze',

```

```
#stopword removal
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
filtered_words = [w for w in words if w.lower() not in stop_words]

print("Words after stopword removal:", filtered_words)
```

ter stopword removal: ['Natural', 'Language', 'Processing', 'amazing', '.', 'analyze', 'text', 'eas

```
#stemming
from nltk.stem import PorterStemmer

ps = PorterStemmer()
stemmed_words = [ps.stem(w) for w in words]

print("Stemmed Words:", stemmed_words)
```

Stemmed Words: ['natur', 'languag', 'process', 'is', 'amaz', '.', 'you', 'can', 'analyz', 'text',

```
#lemmatization
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(w) for w in words]

print("Lemmatized Words:", lemmatized_words)
```

Lemmatized Words: ['Natural', 'Language', 'Processing', 'is', 'amazing', '.', 'You', 'can', 'analy

```
#part of speech (POS)
pos_tags = nltk.pos_tag(words)
print("POS Tags:", pos_tags)
```

POS Tags: [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'VBZ'), ('amazing

```
#N-gram (unigrams,bigrams,trigrams)
```

```
from nltk.util import ngrams

bigrams = list(ngrams(words, 2))
trigrams = list(ngrams(words, 3))

print("Bigrams:", bigrams)
print("Trigrams:", trigrams)
```

Bigrams: [('Natural', 'Language'), ('Language', 'Processing'), ('Processing', 'is'), ('is', 'amazi
Trigrams: [('Natural', 'Language', 'Processing'), ('Language', 'Processing', 'is'), ('Processing',

```
#name entity recognition
nltk.download('maxent_ne_chunker')
nltk.download('words')
```

```
from nltk import ne_chunk
```

```
ner_tree = ne_chunk(pos_tags)
print(ner_tree)
```

```
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data] /root/nltk_data...
[nltk_data] Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Package words is already up-to-date!
(S
  Natural/JJ
  Language/NNP
  Processing/NNP
  is/VBZ
  amazing/VBG
  ./
  You/PRP
  can/MD
  analyze/VB
  text/JJ
  easily/RB
  !/.)
```

```
#bag of words
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = [
    "Natural language processing is fun",
    "I love learning NLP"
]
```

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
```

```
print("Vocabulary:", vectorizer.get_feature_names_out())
print("Bag-of-Words Matrix:\n", X.toarray())
```

```
Vocabulary: ['fun' 'is' 'language' 'learning' 'love' 'natural' 'nlp' 'processing']
Bag-of-Words Matrix:
[[1 1 1 0 0 1 0 1]
 [0 0 0 1 1 0 1 0]]
```

```
#TF-IDF vectorization
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(corpus)
```

```
print("Vocabulary:", tfidf.get_feature_names_out())
print("TF-IDF Matrix:\n", X_tfidf.toarray())
```



```
Vocabulary: ['fun' 'is' 'language' 'learning' 'love' 'natural' 'nlp' 'processing']
TF-IDF Matrix:
[[0.4472136  0.4472136  0.4472136  0.          0.          0.4472136
  0.          0.4472136 ]
 [0.          0.          0.          0.57735027 0.57735027 0.
  0.57735027 0.          ]]
```

```
!pip install gensim
import gensim
from gensim.utils import simple_preprocess
from gensim import corpora
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
27.9/27.9 MB 49.7 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
text = "Natural Language Processing with Gensim is powerful and efficient."

tokens = simple_preprocess(text)
print("Tokens:", tokens)
```

```
Tokens: ['natural', 'language', 'processing', 'with', 'gensim', 'is', 'powerful', 'and', 'efficient']
```

```
from gensim.parsing.preprocessing import STOPWORDS

filtered_tokens = [w for w in tokens if w not in STOPWORDS]
print("Filtered Tokens:", filtered_tokens)
```

```
Filtered Tokens: ['natural', 'language', 'processing', 'gensim', 'powerful', 'efficient']
```

```
texts = [
    simple_preprocess("I love natural language processing"),
    simple_preprocess("Gensim makes text processing simple"),
    simple_preprocess("NLP is fun with python")
]

# Create dictionary
dictionary = corpora.Dictionary(texts)
print("Dictionary:", dictionary.token2id)
```

```
Dictionary: {'language': 0, 'love': 1, 'natural': 2, 'processing': 3, 'gensim': 4, 'makes': 5, 'si
```

```
# Convert corpus into BoW format
corpus_bow = [dictionary.doc2bow(text) for text in texts]
print("BoW Corpus:", corpus_bow)
```

```
BoW Corpus: [[(0, 1), (1, 1), (2, 1), (3, 1)], [(3, 1), (4, 1), (5, 1), (6, 1), (7, 1)], [(8, 1),
```

```
from gensim.models import TfidfModel
```

```
tfidf = TfidfModel(corpus_bow)
corpus_tfidf = tfidf[corpus_bow]
```

```
print("TF-IDF Corpus:")
for doc in corpus_tfidf:
    print(doc)
```

```
TF-IDF Corpus:
[(0, np.float64(0.5646732768699807)), (1, np.float64(0.5646732768699807)), (2, np.float64(0.5646
[(3, np.float64(0.18147115159841573)), (4, np.float64(0.49169813431045906)), (5, np.float64(0.49
[(8, np.float64(0.4472135954999579)), (9, np.float64(0.4472135954999579)), (10, np.float64(0.44721
```



```
from gensim.models import Word2Vec
```

```
model = Word2Vec(sentences=texts, vector_size=100, window=5, min_count=1, workers=4)
```

```
# Word vector
print(model.wv['gensim'])
```

```
# Most similar words
print(model.wv.most_similar('processing'))
```

```
[-0.00515624 -0.00666834 -0.00777684  0.00831073 -0.00198234 -0.00685496
-0.00415439  0.00514413 -0.00286914 -0.00374966  0.00162143 -0.00277629
-0.00158436  0.00107449 -0.00297794  0.00851928  0.00391094 -0.00995886
 0.0062596  -0.00675425  0.00076943  0.00440423 -0.00510337 -0.00211067
 0.00809548 -0.00424379 -0.00763626  0.00925791 -0.0021555  -0.00471943
 0.0085708  0.00428334  0.00432484  0.00928451 -0.00845308  0.00525532
 0.00203935  0.00418828  0.0016979  0.00446413  0.00448629  0.00610452
-0.0032021  -0.00457573 -0.00042652  0.00253373 -0.00326317  0.00605772
 0.00415413  0.00776459  0.00256927  0.00811668 -0.00138721  0.00807793
 0.00371702 -0.00804732 -0.00393361 -0.00247188  0.00489304 -0.00087216
-0.00283091  0.00783371  0.0093229  -0.00161493 -0.00515925 -0.00470176
-0.00484605 -0.00960283  0.00137202 -0.00422492  0.00252671  0.00561448
-0.00406591 -0.00959658  0.0015467  -0.00670012  0.00249517 -0.00378063
 0.00707842  0.00064022  0.00356094 -0.00273913 -0.00171055  0.00765279
 0.00140768 -0.00585045 -0.0078345  0.00123269  0.00645463  0.00555635
-0.00897705  0.00859216  0.00404698  0.00746961  0.00974633 -0.00728958
-0.00903996  0.005836  0.00939121  0.00350693]
[('gensim', 0.21617142856121063), ('love', 0.0931011214852333), ('text', 0.09291722625494003), ('r
```

```
from gensim.models import LdaModel
```

```
lda_model = LdaModel(corpus=corpus_bow, id2word=dictionary, num_topics=3, passes=10)
```

```
# Print topics
topics = lda_model.print_topics(num_words=5)
for topic in topics:
```

```
print(topic)
```

```
(0, '0.175*"processing" + 0.100*"natural" + 0.100*"love" + 0.100*"language" + 0.100*"makes"')
(1, '0.143*"python" + 0.143*"is" + 0.143*"fun" + 0.143*"with" + 0.143*"nlp"')
(2, '0.077*"simple" + 0.077*"processing" + 0.077*"text" + 0.077*"gensim" + 0.077*"makes"')
```

```
from gensim.similarities import MatrixSimilarity

index = MatrixSimilarity(tfidf[corpus_bow])

query = simple_preprocess("natural language processing with python")
query_bow = dictionary.doc2bow(query)
query_tfidf = tfidf[query_bow]

similarities = index[query_tfidf]
print("Similarity scores:", similarities)
```

WARNING:gensim.similarities.docsim:scanning corpus to determine the number of features (consider
Similarity scores: [0.59311694 0.03293178 0.4397882]



```
# from gensim.summarization import summarize # This module has been removed in Gensim 4.x
```

```
large_text = """
Natural Language Processing allows computers to understand human language.
Gensim helps with topic modeling, document similarity, and vectorization.
It is widely used for text analysis and NLP tasks.
"""
```

```
# print(summarize(large_text, ratio=0.5)) # Summarization functionality is no longer directly available
print("Gensim's summarization module was removed in version 4.0. Please consider alternative libraries for summarization")
```

```
Gensim's summarization module was removed in version 4.0. Please consider alternative libraries for summarization
```